

Exercise 6 Semaphore

Learning Objectives:

- Recognize critical sections
- Differentiate between named and unnamed semaphores
- Understand and explain functions for using semaphores
- Secure critical sections with appropriate countermeasures

Legend:

 Reading that must be done before the exercise

 Questions/tasks to be completed before the exercise

 Tasks to be completed during the exercise

6.1 Preparations for the Exercise

The preparations and questions should be completed before the exercise session. The exception is this first exercise, as you have not had a chance to read the material yet. These are also intended to help you with the tasks to be completed during the exercise sessions.

 Study the following materials in preparation:

- Chapter 9.4 in Christian Baun, "Operating Systems - Bilingual Edition", Springer Vieweg, 2nd edition
- `sem_overview(7)` Linux Manual Page Online

 **Introductory Questions:**

a) What is a critical section? Describe in keywords.

b) What is the difference between named and unnamed semaphores? In the man pages for `sem_overview` there is an explanatory overview.

c) The following functions are important for so-called *named* POSIX semaphores:

- `sem_open` Establish a connection or create a *named* semaphore
- `sem_unlink` Terminate a connection to a semaphore or remove a semaphore from the system when the last process terminates the connection.
- `sem_close` Release the (memory) resources for a semaphore and remove the semaphore when the last process terminates the connection.
- `sem_wait` Wait for a semaphore, corresponds to the P-operation
- `sem_post` Release a semaphore, corresponds to the V-operation

Learn about the overview `sem_overview` and the four functions either through man pages or/and on the internet. Describe all five functions in your own words and provide explanations on which parameters are used to call the functions.

- `sem_open`

- `sem_unlink`

- `sem_close`

- `sem_wait`

- `sem_post`

- d) How can you determine on a Linux system whether a *named* POSIX semaphore has been created?

- e) Review the C program provided in Exercise 6.2 and describe its behavior in a few sentences. Specifically, the creation of the parent and child processes should be elaborated upon. The role of the semaphore should be described during the exercise.

 Important at the beginning:

Keep in mind that everyone can log into the Raspberry Pi. Therefore, ensure that all personal data is copied to a personal area at the latest before finishing.

6.2 Semaphore for Parent-Child Process Synchronization

❖ The following C program in Listing 1 is provided, which you are already familiar with from the previous exercise. Access to the critical section should now be regulated using a semaphore.

Listing 1: posix_semaphore_example.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <semaphore.h>

#include <fcntl.h>

const char *semName = "mysemaphore";

void parent(void)
{
    sem_t *sem_id = sem_open(semName, O_CREAT, 0600, 0);

    if (sem_id == SEM_FAILED)
    {
        /* No connection to semaphore acquired */
        perror("Parent : [sem_open] Failed\n");
        exit(EXIT_FAILURE);
    }

    printf("Parent : Wait for Child to Print\n");
    if (sem_wait(sem_id) < 0)
    {
        printf("Parent : [sem_wait] Failed\n");
    }
    printf("Parent : Child Printed! \n");

    if (sem_close(sem_id) != 0)
```

```
{
    perror("Parent : [sem_close] Failed\n");
    exit(EXIT_FAILURE);
}

if (sem_unlink(semName) < 0)
{
    printf("Parent : [sem_unlink] Failed\n");
    exit(EXIT_FAILURE);
}
}

void child(void)
{
    sem_t *sem_id = sem_open(semName, O_CREAT, 0600, 0);

    if (sem_id == SEM_FAILED)
    { /* No connection to semaphore acquired */
        perror("Child : [sem_open] Failed\n");
        exit(EXIT_FAILURE);
    }

    printf("Child : I am done! Release Semaphore\n");
    if (sem_post(sem_id) < 0)
    {
        printf("Child : [sem_post] Failed \n");
    }
}

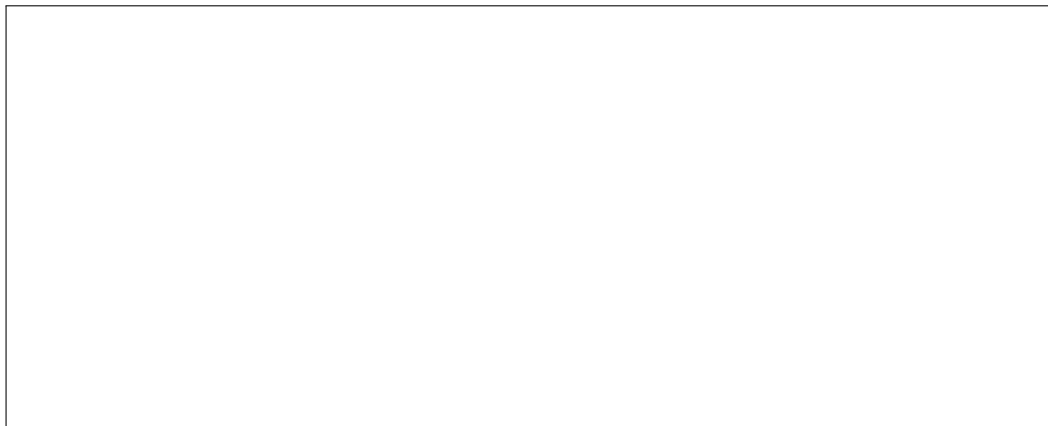
int main(int argc, char *argv[])
{
    pid_t pid;
    pid = fork();

    if (pid < 0){
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0)
    { /* child process */
        child();
        printf("Child : Done with sem_open \n");
    }
}
```

```
}  
else  
{ /* parent process */  
    int status;  
    parent();  
    wait(&status);  
    printf("Parent    : Done with sem_open \n");  
}  
  
return 0;  
}
```

- a) Compile the C program either in Eclipse or using the command **make**. The corresponding **makefile** has been provided with this exercise.
- b) What is the purpose of the semaphore used in this program? Describe the workflow using a diagram and keywords.



6.3 Semaphore to Prevent Race Conditions

The example program `uebung05_2_race_posix.c` from the previous exercise highlighted the *race condition* between various child processes. Now use a **named** semaphore to secure the critical section, ensuring that it is always accessible to only one process at a time. **Note:** You may need to modify some program parts slightly, but maintain the functionality.

- a) How many semaphores do you need?



- b) Which function calls are suitable for creating, securing, and deleting the semaphore, and at what points should they be placed?



- c) Modify the example program accordingly, save it as

`uebung06_3_semaphore_race_bsp.c`

compile it, and test it.

- d) Verify whether the creation of semaphores works on your Linux system. This should have already been clarified beforehand.